

Práctica 2

Calculadora de expresiones aritméticas mediante sockets

1. Objetivo

Desarrollar una aplicación distribuida bajo el modelo **cliente-servidor**, utilizando comunicación mediante **sockets**, que permita al cliente introducir expresiones aritméticas a través de una interfaz gráfica y enviarlas al servidor para su procesamiento y evaluación.

2. Descripción del proyecto

2.1. Del lado del cliente

La aplicación cliente deberá proporcionar una **interfaz gráfica** mediante la cual el usuario pueda introducir una expresión aritmética.

La expresión podrá contener:

- Números enteros y/o decimales.
- Operador suma (+).
- Operador resta (-).
- Operador multiplicación (*).
- Operador división (/).
- Paréntesis para agrupación (()).

Ejemplos de expresiones válidas:

```
5 + 8
10 - 3 * 2
(10 + 5) * 2
25 / (5 + 5)
(8 + 2) * (15 - 5)
10 + (20 / 5) - 3
```

El **cliente** será responsable de:

1. Capturar la expresión mediante la interfaz gráfica.
2. Validar que exista una expresión antes de enviarla.
3. Establecer la conexión con el servidor.
4. Enviar la expresión al servidor.
5. Recibir la respuesta.
6. Mostrar el resultado o el mensaje de error recibido en la pantalla de la Interfaz gráfica.

2.2. Aplicación servidor

El servidor deberá permanecer a la espera de conexiones provenientes de los clientes.

Cuando reciba una expresión, deberá:

1. Recibir la expresión enviada por el cliente.
2. Validar la expresión.
3. Evaluar la expresión respetando la precedencia de operadores y el uso de paréntesis.
4. **Detectar errores de sintaxis o de operación.**
5. Generar una respuesta con el resultado o el mensaje correspondiente.
6. Enviar la respuesta al cliente.

Por ejemplo:

Solicitud enviada por el cliente:

```
(15 + 5) * 3
```

Respuesta del servidor:

```
Resultado: 60
```

En caso de una operación no válida:

Solicitud:

```
10 / 0
```

Respuesta:

```
Error: no es posible realizar una división entre cero.
```

3. Requisitos del servidor

El servidor deberá cumplir, como mínimo, con las siguientes características:

- Utilizar comunicación mediante sockets.
- Escuchar conexiones en un puerto determinado.
- Aceptar conexiones provenientes de clientes.
- Recibir expresiones aritméticas.
- Procesar y evaluar las expresiones recibidas.
- Considerar la precedencia de operadores.
- Resolver correctamente expresiones que utilicen paréntesis.
- Detectar errores de sintaxis.
- Detectar división entre cero.
- Enviar al cliente el resultado o el mensaje de error correspondiente.
- Mostrar en consola información básica de las conexiones y solicitudes recibidas.
- Permitir finalizar el servidor de manera controlada.

Como extensión, el servidor podrá permitir la atención de **múltiples clientes**, utilizando hilos o algún mecanismo de concurrencia (Computadoras de integrantes del mismo equipo).

4. Requisitos del cliente

El cliente deberá contar con una interfaz gráfica que incluya, como mínimo:

- Botones numéricos y de operadores en la interfaz
- Campo de texto para introducir la expresión.
- Botón para enviar/evaluar la expresión (Signo =).
- Área para mostrar el resultado.
- Indicador del estado de conexión con el servidor (Botón o indicador activado).
- Opción para limpiar la expresión y/o el resultado.
- Mensajes adecuados cuando no exista conexión con el servidor.
- Mensajes adecuados cuando el servidor reporte un error.
- Historial de operaciones.

5. Operaciones y expresiones a soportar

La aplicación deberá soportar, como **mínimo**, los siguientes operadores:

Operador	Operación	Ejemplo
+	Suma	$8 + 5$
-	Resta	$8 - 5$
*	Multipliación	$8 * 5$
/	División	$8 / 5$
()	Agrupación	$(8 + 5) * 2$

La evaluación deberá respetar la precedencia convencional de las operaciones.

Por ejemplo:

$$2 + 3 * 4$$

deberá interpretarse como:

$$2 + (3 * 4)$$

y no como:

$$(2 + 3) * 4$$

Asimismo:

$$(2 + 3) * 4$$

deberá evaluarse primero dentro del paréntesis.

6. Manejo de errores

El sistema deberá contemplar, como mínimo, los siguientes casos:

- Expresión vacía (al dar clic en el signo = sin ninguna expresión capturada).
- Expresión con caracteres no permitidos (Signos especiales u operadores no válidos).
- Paréntesis no balanceados.

- Operadores colocados incorrectamente.
- Expresión incompleta. (Sin operadores)
- División entre cero.
- Error al establecer la conexión con el servidor.
- Desconexión del servidor.
- Tiempo de espera agotado para la respuesta.

Los errores deberán comunicarse al usuario mediante mensajes claros y comprensibles.

7. Tecnologías a utilizar

La práctica podrá desarrollarse utilizando las tecnologías que domines y de tu preferencia. Por ejemplo:

Lenguaje de programación:

[Java / Python / C# / otro]

Interfaz gráfica:

[Java Swing / JavaFX / Tkinter / Windows Forms / otro]

Comunicación:

Sockets utilizando el protocolo **TCP**.

Arquitectura:

Modelo Cliente-Servidor.

Herramientas de desarrollo:

[IDE utilizado]

8. Casos de pruebas

Se deberán realizar varios casos de prueba con diferentes tipos de expresiones, por citar algunos ejemplos:

Prueba	Expresión	Resultado esperado
1	5 + 3	8
2	10 - 7	3
3	4 * 6	24
4	20 / 4	5
5	2 + 3 * 4	14
6	(2 + 3) * 4	20
7	10 / (2 + 3)	2
8	(10 - 2) * (3 + 1)	32
9	10 / 0	Error
10	(5 + 3	Error
11	5 + * 3	Error
12	expresión vacía	Error

Además, se deberán incluir evidencias de:

- Servidor iniciado y escuchando conexiones.
- Cliente conectado al servidor.
- Envío de una expresión.
- Resultado correcto de una operación.
- Evaluación de una expresión con paréntesis.
- Manejo de una expresión incorrecta.
- Manejo de división entre cero.
- Desconexión o ausencia del servidor.

Los resultados de las pruebas deberán documentarse mediante capturas de pantalla o evidencias de ejecución.

9. Productos entregables

Se deberá entregar:

9.1. Código fuente

Repositorio o carpeta que contenga:

- Código fuente del servidor.
- Código fuente del cliente.
- Archivos correspondientes a la interfaz gráfica.
- Archivos de configuración necesarios.
- Incluir un archivo README con instrucciones de compilación y/o ejecución.

9.2. Reporte de la práctica

El documento de acuerdo a los puntos compartidos en la Práctica 1.

9.3. Video de demostración

Entregar un video breve donde se muestre:

1. Inicio del servidor.
2. Inicio del cliente.
3. Conexión entre ambos.
4. Evaluación de expresiones simples.
5. Evaluación de expresiones con paréntesis.
6. Manejo de errores.
7. Comunicación multi usuario
8. Cierre de las aplicaciones.